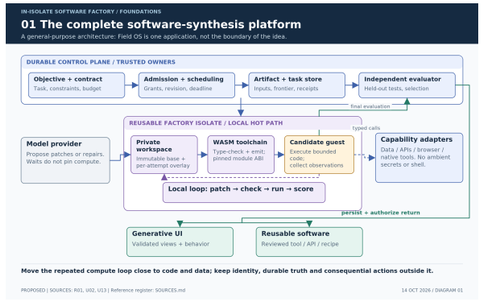
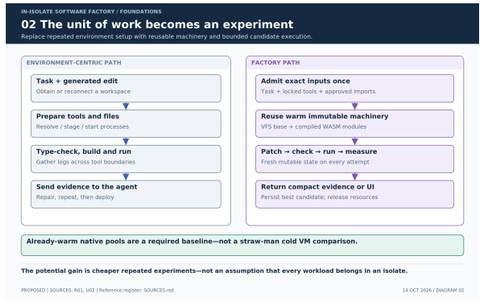
Diagram atlas / visual index

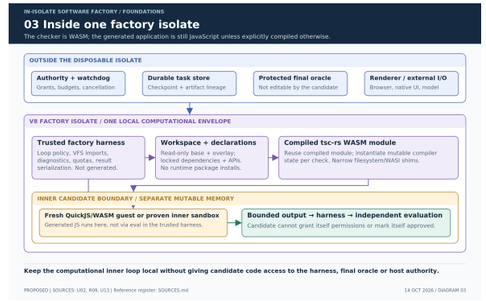
Sheets 01–15 | Click any thumbnail to open its full-size vector page.



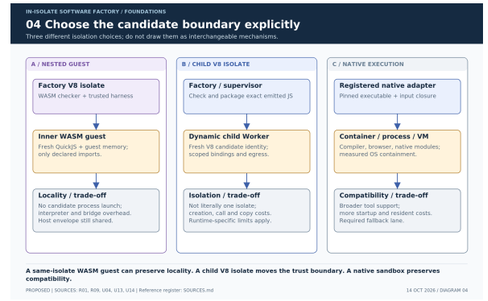
01 The complete software-synthesis platform



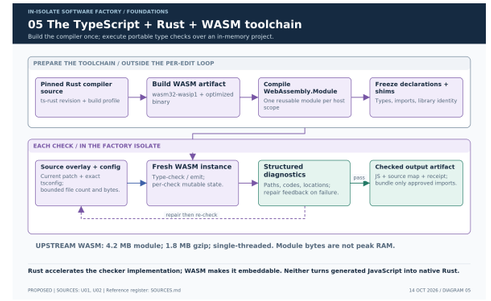
02 The unit of work becomes an experiment



03 Inside one factory isolate



04 Choose the candidate boundary explicitly



05 The TypeScript + Rust + WASM toolchain

06 One candidate, from request to presentation

07 Three lifetimes, three storage rules

08 Count the full resource envelope

09 Agentic coding: make code the tool

10 Swarm hill-climbing with a bounded frontier

11 Evaluation is a ladder, not a single score

12 Generative UI is a tested software artifact

13 Optimize UI behavior through measured loops

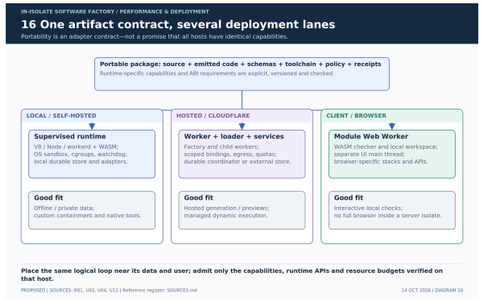
14 Keep the interface stable while candidates evolve

15 Where speed and density can come from

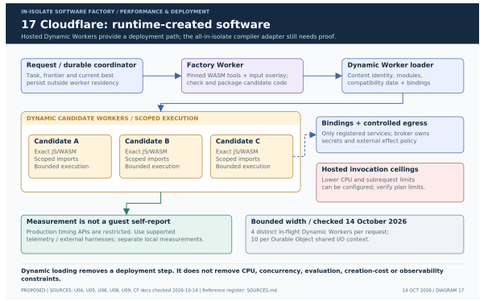
Blue: authority | Purple: reusable tools | Amber: candidate execution | Green: evidence | Red: failure/risk

Diagram atlas / visual index

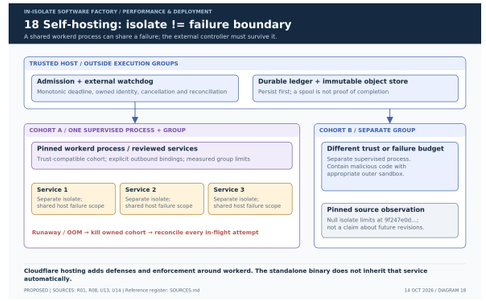
Sheets 16–30 | Click any thumbnail to open its full-size vector page.



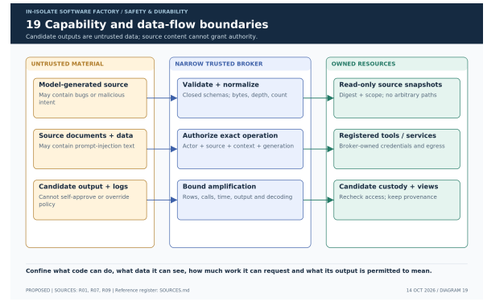
16 One artifact contract, several deployment lanes



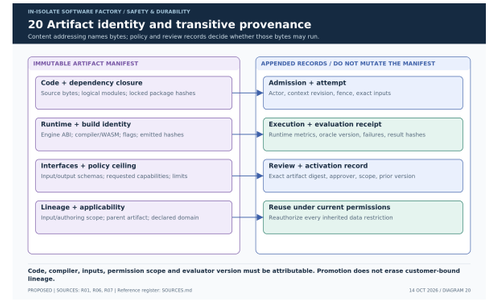
17 Cloudflare: runtime-created software



18 Self-hosting: isolate != failure boundary



19 Capability and data-flow boundaries



20 Artifact identity and transitive provenance

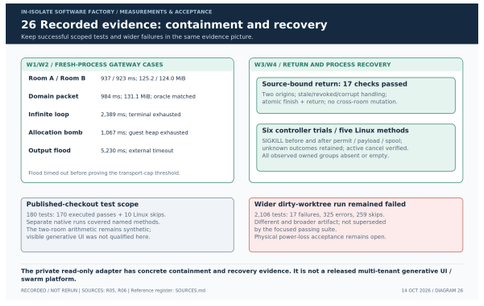
21 Task lifecycle and attempt boundaries

22 Durable dispatch and crash reconciliation

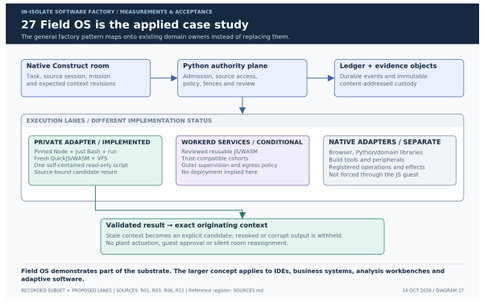
23 A swarm is a queue, not a pile of idle machines

24 Recorded evidence: warm execution

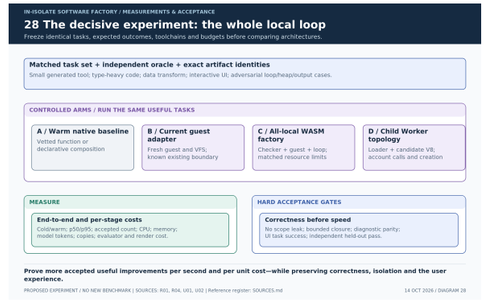
25 Recorded evidence: native tsc-rs gains



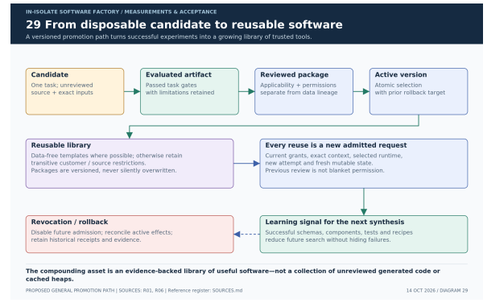
26 Recorded evidence: containment and recovery



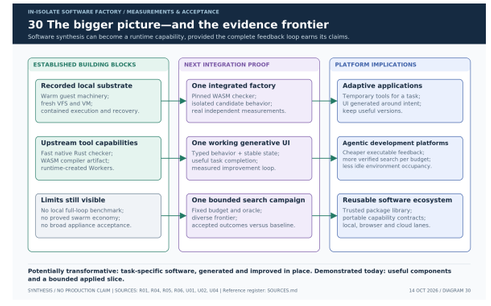
27 Field OS is the applied case study



28 The decisive experiment: the whole local loop



29 From disposable candidate to reusable software

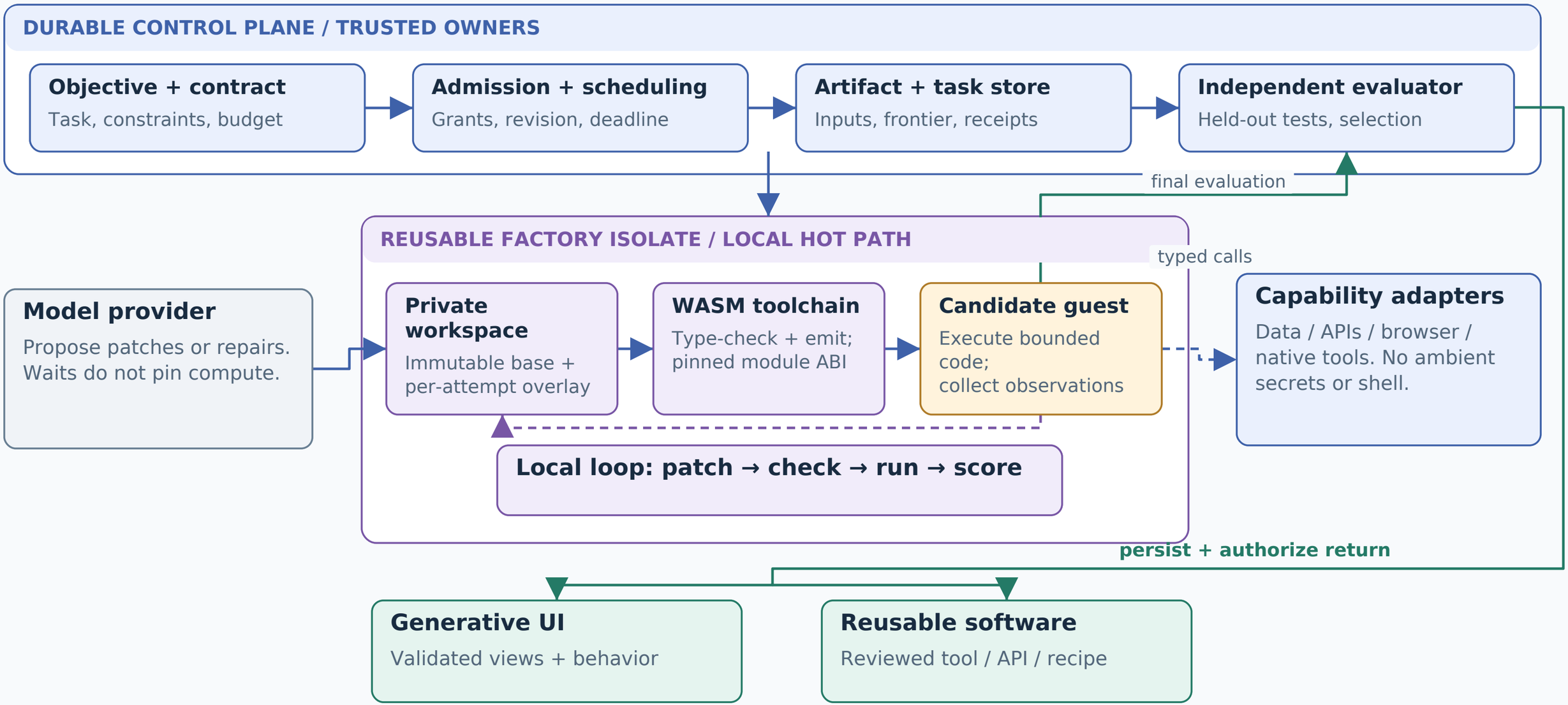


30 The bigger picture—and the evidence frontier

Blue: authority | Purple: reusable tools | Amber: candidate execution | Green: evidence | Red: failure/risk

01 The complete software-synthesis platform

A general-purpose architecture: Field OS is one application, not the boundary of the idea.



Move the repeated compute loop close to code and data; keep identity, durable truth and consequential actions outside it.

02 The unit of work becomes an experiment

Replace repeated environment setup with reusable machinery and bounded candidate execution.

ENVIRONMENT-CENTRIC PATH

Task + generated edit
Obtain or reconnect a workspace

↓

Prepare tools and files
Resolve / stage / start processes

↓

Type-check, build and run
Gather logs across tool boundaries

↓

Send evidence to the agent
Repair, repeat, then deploy

FACTORY PATH

Admit exact inputs once
Task + locked tools + approved imports

↓

Reuse warm immutable machinery
VFS base + compiled WASM modules

↓

Patch → check → run → measure
Fresh mutable state on every attempt

↓

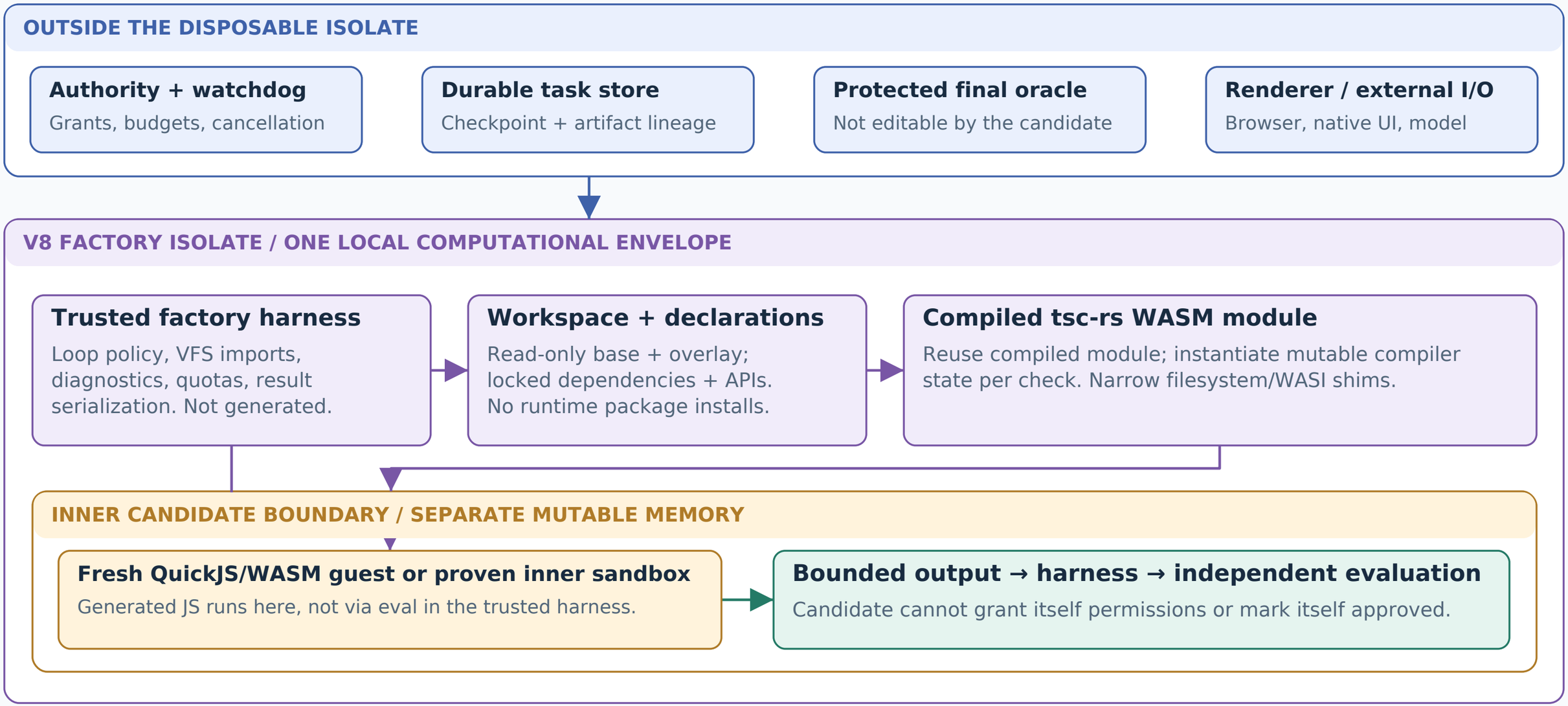
Return compact evidence or UI
Persist best candidate; release resources

Already-warm native pools are a required baseline—not a straw-man cold VM comparison.

The potential gain is cheaper repeated experiments—not an assumption that every workload belongs in an isolate.

03 Inside one factory isolate

The checker is WASM; the generated application is still JavaScript unless explicitly compiled otherwise.



Keep the computational inner loop local without giving candidate code access to the harness, final oracle or host authority.

04 Choose the candidate boundary explicitly

Three different isolation choices; do not draw them as interchangeable mechanisms.

A / NESTED GUEST

Factory V8 isolate

WASM checker + trusted harness

↓

Inner WASM guest

Fresh QuickJS + guest memory;
only declared imports.

↓

Locality / trade-off

No candidate process launch;
interpreter and bridge overhead.
Host envelope still shared.

B / CHILD V8 ISOLATE

Factory / supervisor

Check and package exact emitted JS

↓

Dynamic child Worker

Fresh V8 candidate identity;
scoped bindings and egress.

↓

Isolation / trade-off

Not literally one isolate;
creation, call and copy costs.
Runtime-specific limits apply.

C / NATIVE EXECUTION

Registered native adapter

Pinned executable + input closure

↓

Container / process / VM

Compiler, browser, native modules;
measured OS containment.

↓

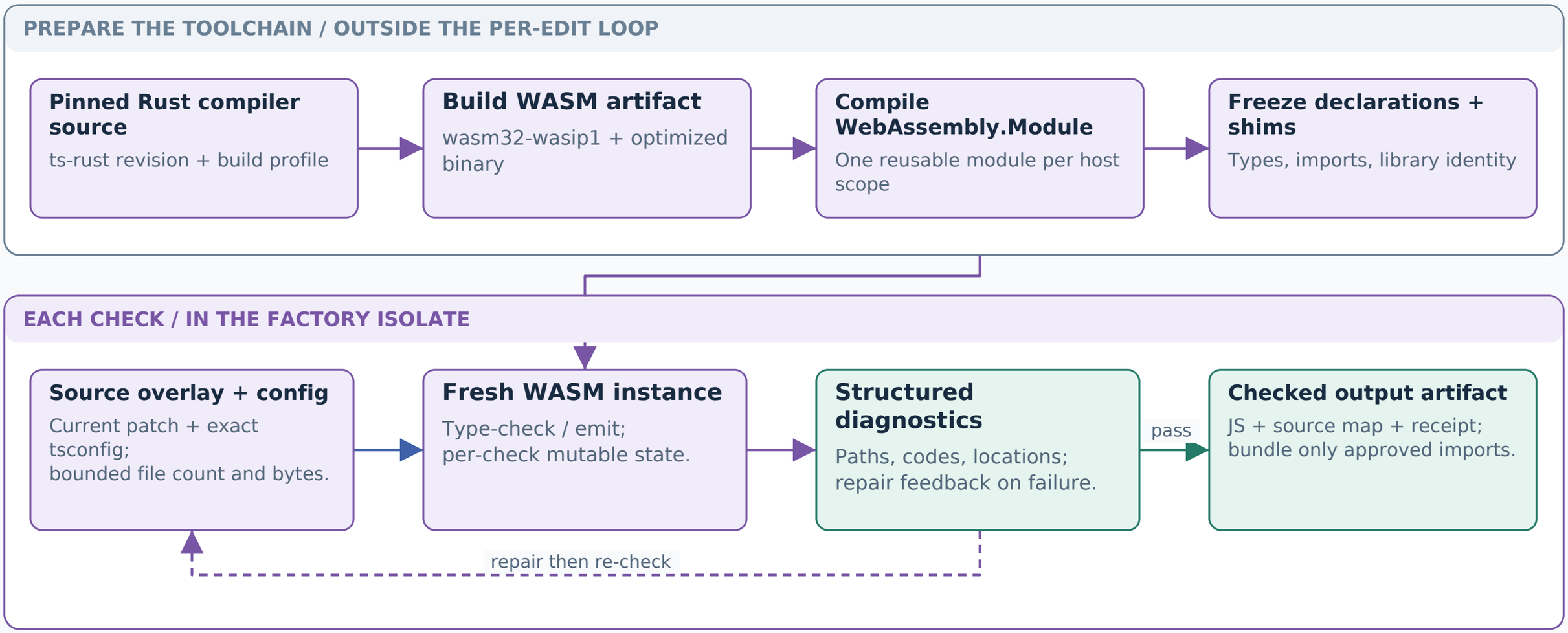
Compatibility / trade-off

Broader tool support;
more startup and resident costs.
Required fallback lane.

A same-isolate WASM guest can preserve locality. A child V8 isolate moves the trust boundary. A native sandbox preserves compatibility.

05 The TypeScript + Rust + WASM toolchain

Build the compiler once; execute portable type checks over an in-memory project.

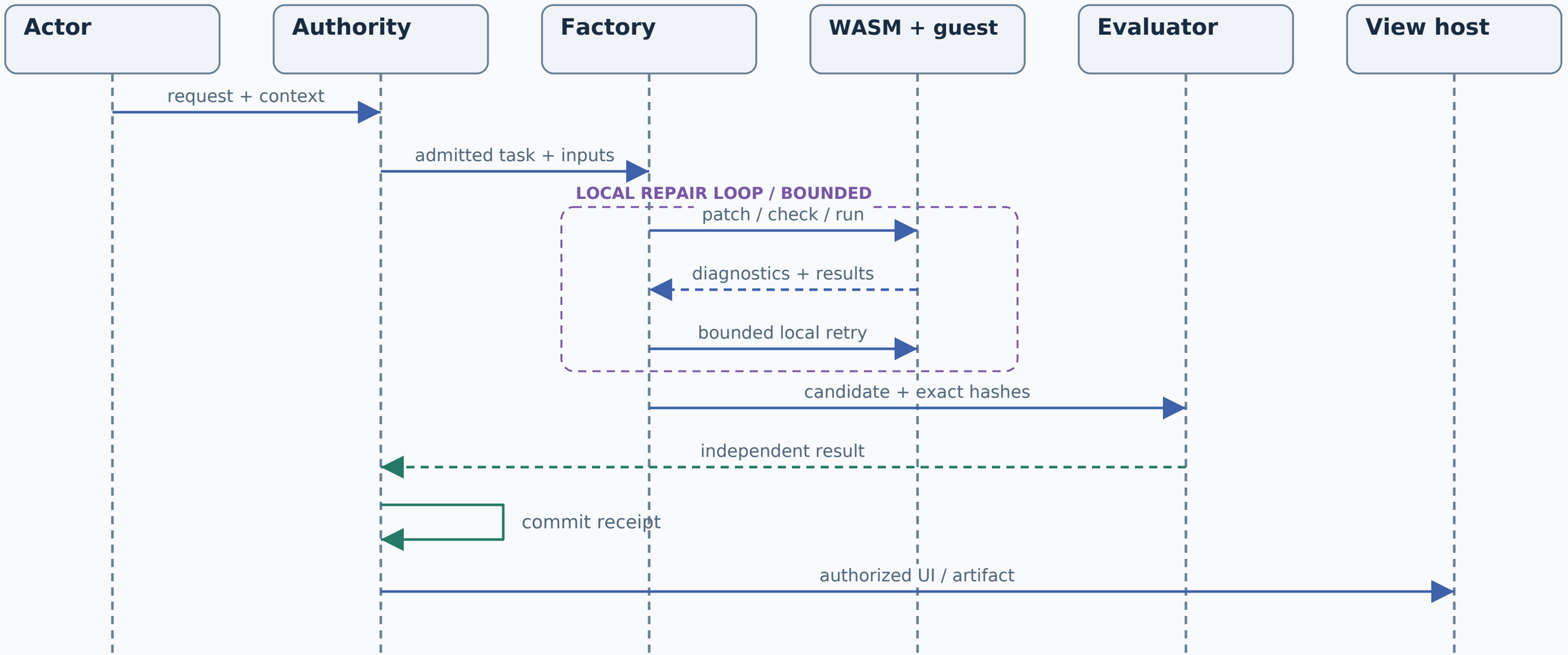


UPSTREAM WASM: 4.2 MB module; 1.8 MB gzip; single-threaded. Module bytes are not peak RAM.

Rust accelerates the checker implementation; WASM makes it embeddable. Neither turns generated JavaScript into native Rust.

06 One candidate, from request to presentation

Sequence diagram: a fast local loop with an external trust and persistence boundary.



Persist the winning result before exposing it as accepted output; intermediate candidates remain replaceable.

07 Three lifetimes, three storage rules

Durable task state, warm immutable caches, and disposable per-attempt state.

DURABLE / correctness depends on retention

Task + objective
Intent, actor, context, budgets

Frontier + current best
Candidate lineage + seed IDs

Objects + receipts
Inputs, outputs, evaluations

WARM / cache loss must be harmless

Compiled WASM module
Engine may reuse compiled machinery

Approved library base
Declarations + immutable package bytes

Read-only input cache
Partitioned by permissions and digest

ATTEMPT / discard after bounded execution

Source + VFS overlay
One candidate, one mutable workspace

Compiler instance + guest
Fresh memory, globals and handles

Transient output buffers
Capped; publish validated results only

Eviction should increase latency, not erase the task or change its truth. Retain references while waiting—not a live guest heap.

08 Count the full resource envelope

Binary size, WASM memory, V8 heap and whole-process memory are different measurements.

EXECUTION GROUP / ACCOUNT THE WHOLE GROUP

Runtime and compiled code

V8 / Node / workerd engine, loaded modules, worker threads and native allocations

WASM linear memories

Compiler state + nested guest; growth, stack and allocator behavior need measurement.

JavaScript heap

Harness, virtual files, strings, parsed objects, promises and result objects.

Bridge and copy buffers

UTF-8 / UTF-16 conversions, IPC frames, serialization, input + output staging.

Host-charged resources

Mapped files, tmpfs, child processes, sockets, tasks and other charged pages.

Enforce aggregate memory, CPU rate, tasks, bytes and wall time

KEEP THESE SEPARATE

Outside the worker group

Supervisor + durable store; model service + GPU memory; renderer / browser; native adapters.

Measure the complete outcome

Useful accepted work / second; CPU-seconds, peak RSS or cgroup; energy and cost if instrumented.

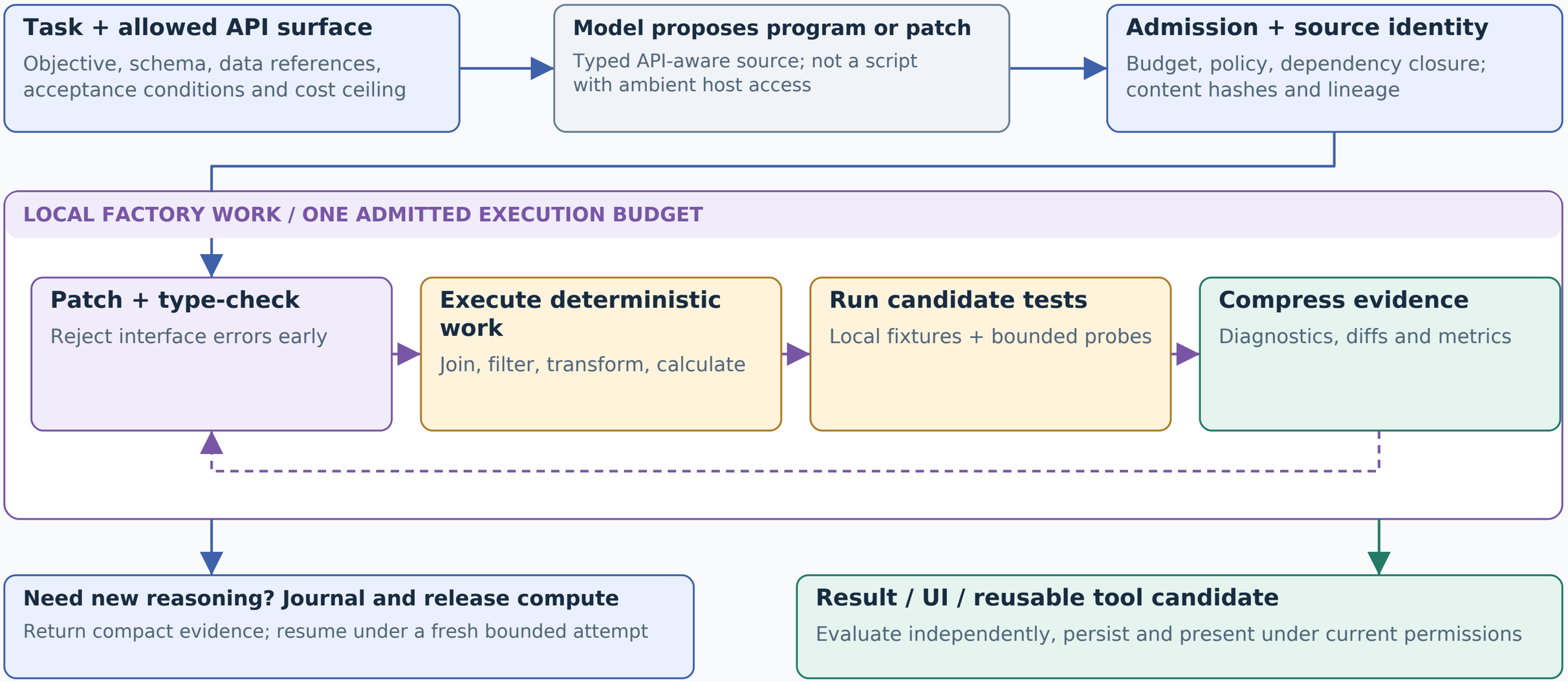
Bound the swarm

Admit against measured headroom; check then run sequentially first; parallelism costs real memory.

Resource efficiency comes from amortized machinery and bounded concurrency—not from treating a 4.2 MB compiler binary as total RAM.

09 Agentic coding: make code the tool

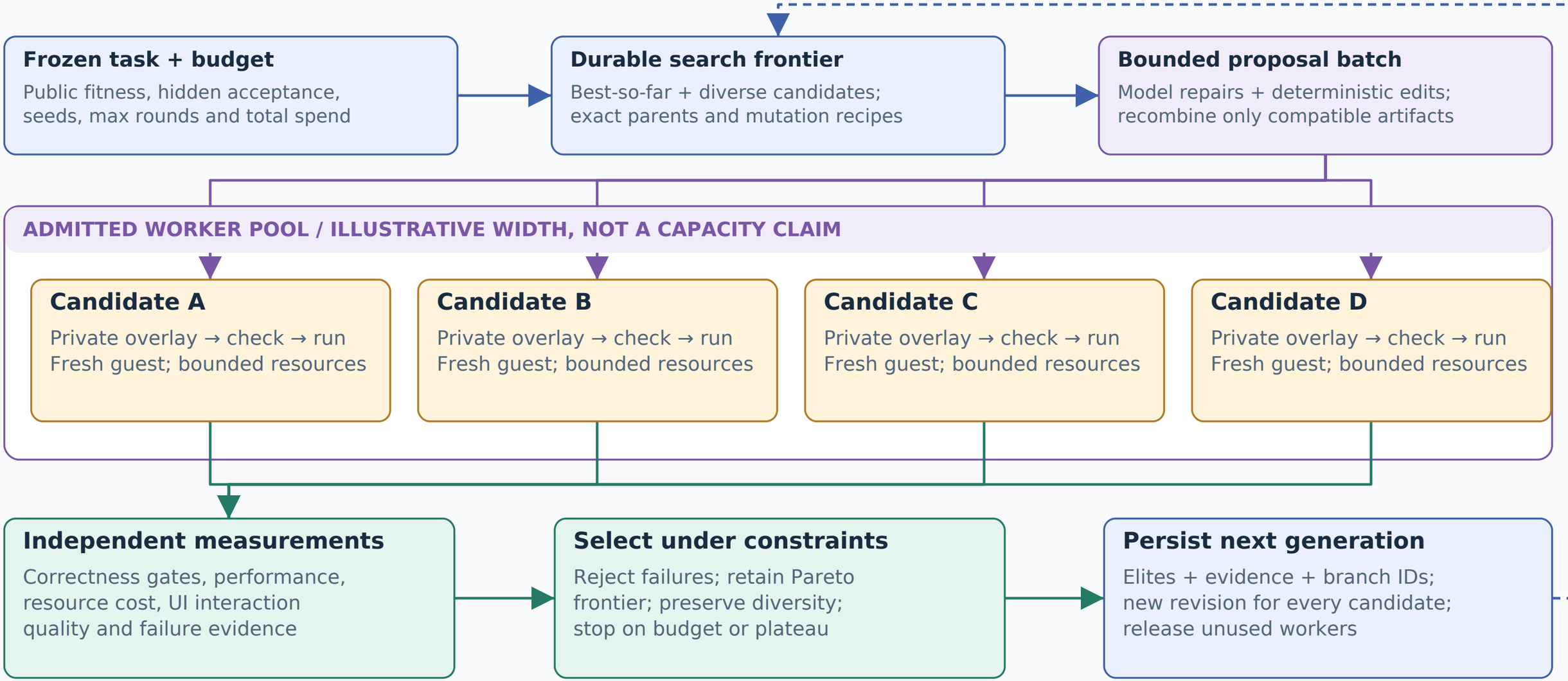
Use models for proposals; use execution for deterministic manipulation, checks and measurements.



The platform can spend model calls on reasoning and repairs while ordinary loops, filters and tests run as bounded software.

10 Swarm hill-climbing with a bounded frontier

Many durable hypotheses; only a measured number of candidates actively consuming compute.



Retain elites and diverse alternatives, not every live runtime. More parallel candidates do not guarantee better solutions.

11 Evaluation is a ladder, not a single score

Cheap deterministic rejection first; expensive independent tests only for survivors.

1 / Contract + syntax
Closed schema; pinned imports; bounded size

2 / Type + interface
Real semantic checks; matching emitted output

3 / Deterministic tests
Unit / property / metamorphic invariants

4 / Interaction + integration
Render, exercise controls, accessibility, APIs

5 / Held-out acceptance
Independent fixtures; blind or rotated trials

6 / Cost + regression gate
Matched baseline; tail latency; memory; stability

any hard failure

PROTECTED EVALUATION PLANE

Hard gates precede ranking
Correctness and permissions are constraints, not soft rewards.

Measure from outside candidate
Trusted time / resource counters; repeat runs; record failures; count setup and inference costs.

Reject or quarantine
Keep bounded reason and lineage.
Do not promote a fast wrong UI.
No test-set leaks in model context.

The candidate may see repair feedback; it must not edit the evaluator, rewrite held-out fixtures or self-report trusted performance.

12 Generative UI is a tested software artifact

Generate layout, behavior and bindings together; the presentation host remains stable.

GENERATED CANDIDATE / VERSIONED PACKAGE

View specification

Registered components, semantic roles, layout and accessibility

Behavior module

Pure transformations, validation and event/state transitions

Data + action bindings

Exact owner / revision; named capability; explicit input schema

Interaction tests + result schema

Expected states, empty/error/stale cases and bounded output

TRUSTED PRESENTATION HOST / ILLUSTRATIVE UI

Validate contract → authorize → reconcile UI

Renderer refuses unknown components, scripts and wider grants

Generated comparison workbench

Source A: pinned revision

Source B: pinned revision

Value / confidence / unit

Evidence remains inspectable

Difference / explanation

Derived, not physical truth

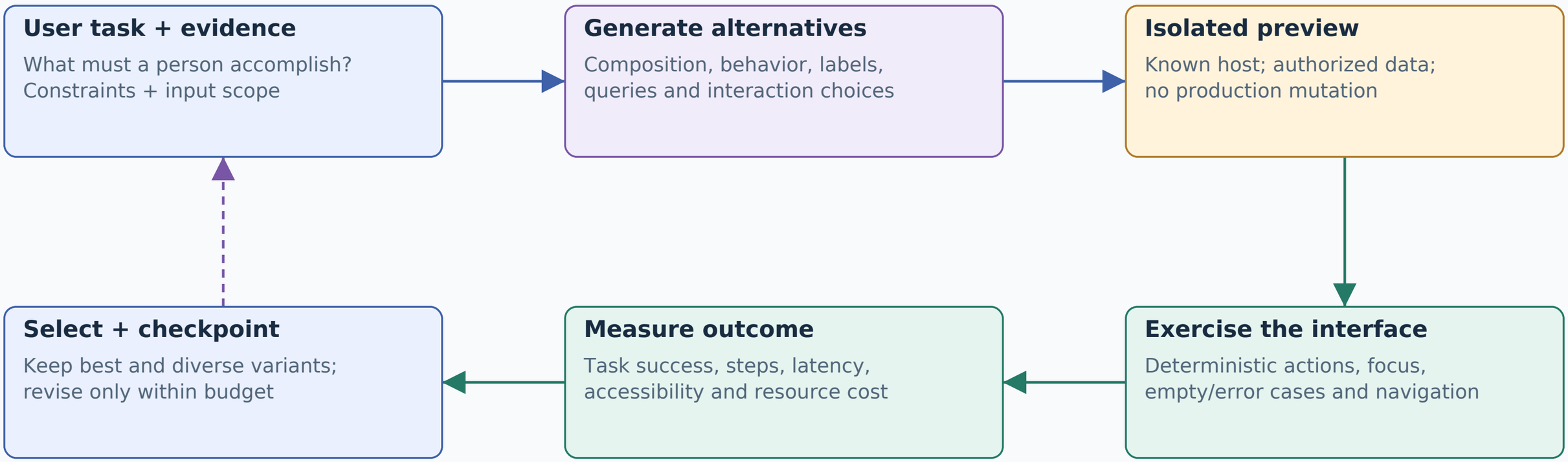
Adjust filter → recompute state

Keep candidate

A useful generated interface is more than an image: it has typed state, working interactions, data lineage and permission-scoped actions.

13 Optimize UI behavior through measured loops

Search over useful interactions—not just visual novelty or an evaluator's first impression.



Human evaluation for genuinely subjective goals

Use consented tasks, declared trade-offs and stable comparison conditions.

Acceptance before activation

Do not use a metric improvement to bypass correctness or permissions.

A generated interface can improve while it runs, but its evaluation needs independent tasks, stable controls and explicit release gates.

14 Keep the interface stable while candidates evolve

Versioned activation, state migration and safe rollback prevent a constantly shifting experience.

ACTIVE VIEW / PINNED V17

Stable instance identity

Inputs, focus, selection, scroll, unsaved edits
and source binding remain owned by host.

Actions target an expected version

Event carries view ID + revision + schema.
Stale events never silently target a new UI.

Rollback target retained

Keep prior bundle + compatible state snapshot.
In-flight effects require reconciliation.

SHADOW CANDIDATE / V18

Generate / check / simulate off-screen

No mutation of active state; no live layout churn.
Compare against the pinned active version.

Validate a state migration

Schema version + deterministic migration;
refuse destructive or incompatible changes.

Atomic activation at a safe boundary

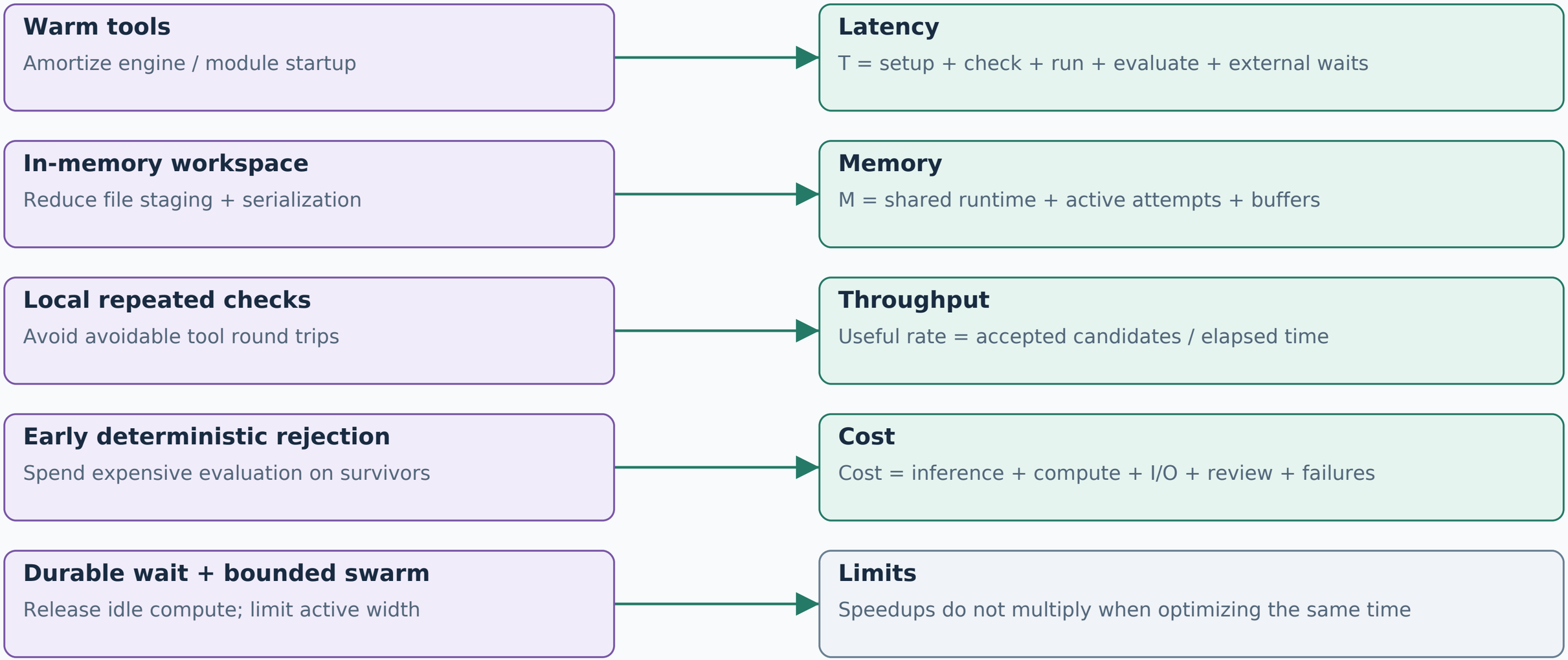
Recheck permissions + source context.
Preserve focus and inputs or require confirmation.

activate / retain prior

The active user experience stays pinned until a validated candidate can replace it without losing input, focus or authority.

15 Where speed and density can come from

A causal model of benefits; these arrows are not an end-to-end benchmark claim.



The economic unit is an independently accepted useful improvement—not a generated token, an isolate count or one fast type check.

16 One artifact contract, several deployment lanes

Portability is an adapter contract—not a promise that all hosts have identical capabilities.

Portable package: source + emitted code + schemas + toolchain + policy + receipts

Runtime-specific capabilities and ABI requirements are explicit, versioned and checked.

LOCAL / SELF-HOSTED

Supervised runtime

V8 / Node / workerd + WASM;
OS sandbox, cgroups, watchdog;
local durable store and adapters.

Good fit

Offline / private data;
custom containment and native tools.

HOSTED / CLOUDFLARE

Worker + loader + services

Factory and child workers;
scoped bindings, egress, quotas;
durable coordinator or external store.

Good fit

Hosted generation / previews;
managed dynamic execution.

CLIENT / BROWSER

Module Web Worker

WASM checker and local workspace;
separate UI main thread;
browser-specific stacks and APIs.

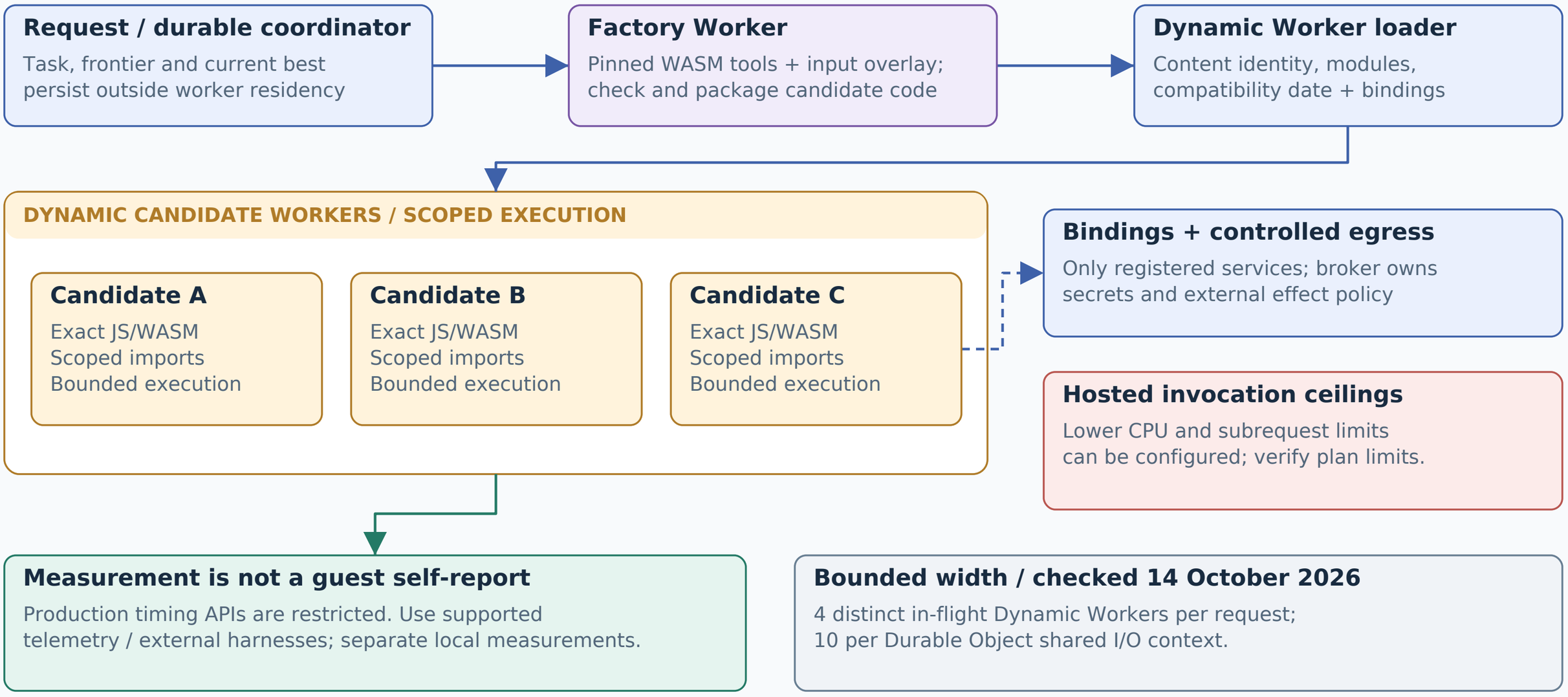
Good fit

Interactive local checks;
no full browser inside a server isolate.

Place the same logical loop near its data and user; admit only the capabilities, runtime APIs and resource budgets verified on that host.

17 Cloudflare: runtime-created software

Hosted Dynamic Workers provide a deployment path; the all-in-isolate compiler adapter still needs proof.



Dynamic loading removes a deployment step. It does not remove CPU, concurrency, evaluation, creation-cost or observability constraints.

18 Self-hosting: isolate != failure boundary

A shared workerd process can share a failure; the external controller must survive it.

TRUSTED HOST / OUTSIDE EXECUTION GROUPS

Admission + external watchdog

Monotonic deadline, owned identity, cancellation and reconciliation

Durable ledger + immutable object store

Persist first; a spool is not proof of completion

COHORT A / ONE SUPERVISED PROCESS + GROUP

Pinned workerd process / reviewed services

Trust-compatible cohort; explicit outbound bindings; measured group limits

Service 1
Separate isolate;
shared host failure scope

Service 2
Separate isolate;
shared host failure scope

Service 3
Separate isolate;
shared host failure scope

Runaway / OOM → kill owned cohort → reconcile every in-flight attempt

COHORT B / SEPARATE GROUP

Different trust or failure budget

Separate supervised process.
Contain malicious code with
appropriate outer sandbox.

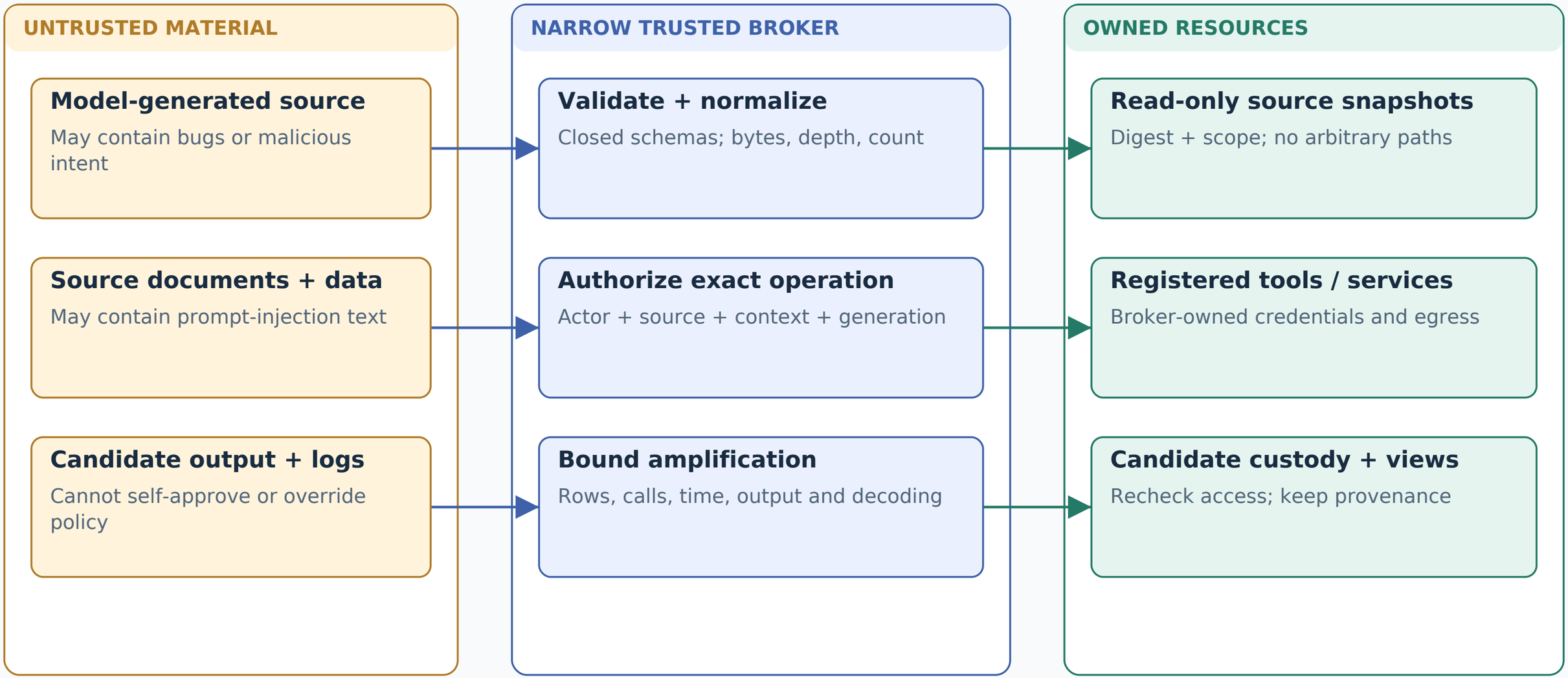
Pinned source observation

Null isolate limits at 9f247e0d...;
not a claim about future revisions.

Cloudflare hosting adds defenses and enforcement around workerd. The standalone binary does not inherit that service automatically.

19 Capability and data-flow boundaries

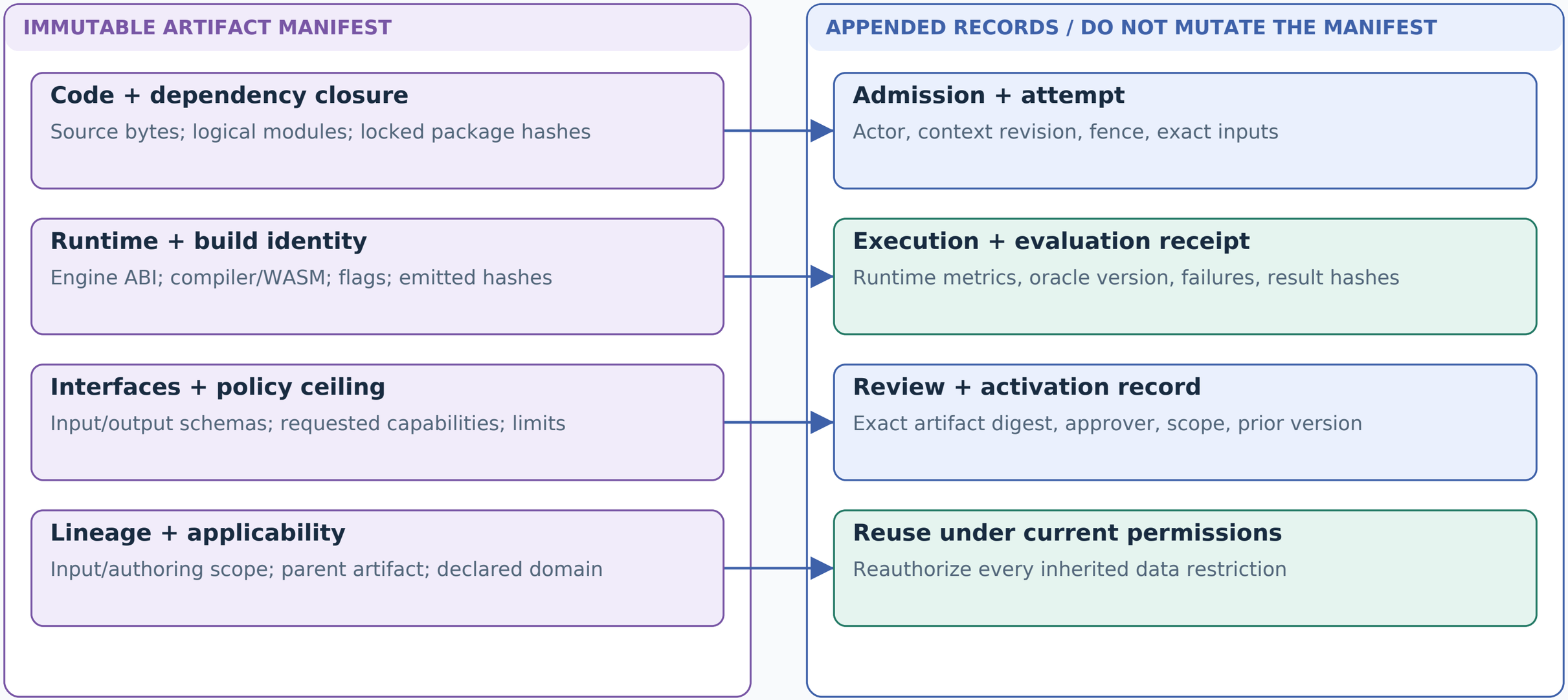
Candidate outputs are untrusted data; source content cannot grant authority.



Confine what code can do, what data it can see, how much work it can request and what its output is permitted to mean.

20 Artifact identity and transitive provenance

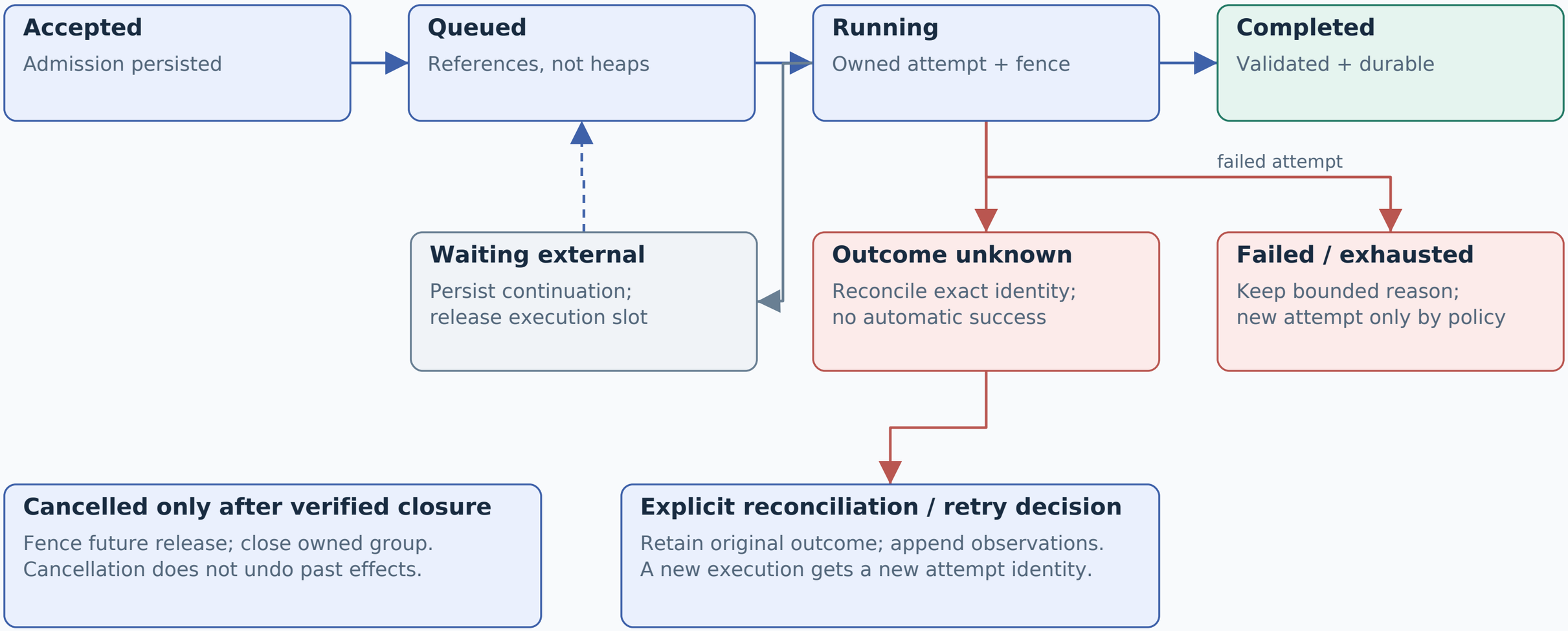
Content addressing names bytes; policy and review records decide whether those bytes may run.



Code, compiler, inputs, permission scope and evaluator version must be attributable. Promotion does not erase customer-bound lineage.

21 Task lifecycle and attempt boundaries

A durable task can outlive many short execution attempts, waits and restarts.



An attempt is potentially executed after authorization, even without a later running record. Never silently retry an ambiguous external effect.

22 Durable dispatch and crash reconciliation

Authorize only the exact verified execution unit; record ambiguity rather than fabricate success.

1 / Commit admission

Request + runtime + source hashes

2 / Claim one attempt

Durable winning fence; no duplicate launch

3 / Start trusted stub

No candidate payload released yet

4 / Verify containment

Namespace + cgroup + nonce + process identity

5 / Persist permit, then release

Potential execution begins at this boundary

6 / Publish result, then commit

Atomic terminal + return; acknowledge afterward

INTERRUPTION RULES

Crash before trusted ownership

Do not adopt an arbitrary unit.
Use bounded passive observation.

Crash after execution permit

Possibly executed. Fence future work; verify exact owned closure; record outcome_unknown when completion unproved.

Crash after successful spool

No canonical finish = not completed.
Keep zero result references until authorized durable completion exists.

A process name, PID alone, or successful leftover file is not sufficient proof of owned execution or durable completion.

23 A swarm is a queue, not a pile of idle machines

Release execution while waiting; use admission to protect foreground responsiveness.

Durable task queue + continuation store

Task IDs, next step, code/input hashes, source grants; frontier checkpoints, priorities and deadlines

Scheduler + admission

Measured memory headroom; CPU and task limits; foreground priority, fair aging and backpressure

ACTIVE COMPUTE SLOTS / EXAMPLE WIDTH ONLY

Active slot 1

Load immutable cache
Fresh attempt state
Check → execute → report

Active slot 2

Load immutable cache
Fresh attempt state
Check → execute → report

Active slot 3

Load immutable cache
Fresh attempt state
Check → execute → report

Waiting / deferred

Model inference
Human review
Network reconnection
External effect reconciliation
No live guest required

Persist compact progress, then free resources

Retain useful candidate, logs within caps, evaluation receipts and continuation references.

Resume = fresh admission checks

Revalidate source scope, context, policy, deadline and runtime; a cache hit is not authorization.

Scale durable agent state independently of active compute. Concurrency is a budgeted resource, not a synonym for intelligence.

24 Recorded evidence: warm execution

Field OS / Main-rig, 8 October 2026. Same six-case synthetic packet; not a V8-versus-VM benchmark.

MEDIAN LATENCY / 100% CPU QUOTA / LOG SCALE



Measurement protocol

5 warm-ups + 20 measured jobs per arm.
Per-job latency excludes host oracle check.
All outputs matched frozen expected bytes.

Whole-unit peak memory

Native: 29.96 MiB
Guest shared VFS: 150.62 MiB
Guest fresh VFS: 151.93 MiB

COLD AND WARM ARE DISTINCT OBSERVATIONS

Cold at 50% CPU quota

Native payload 28 ms; guest payload 669 ms.
Whole supervision: 178 ms vs 814 ms.

Warm at 50% CPU quota

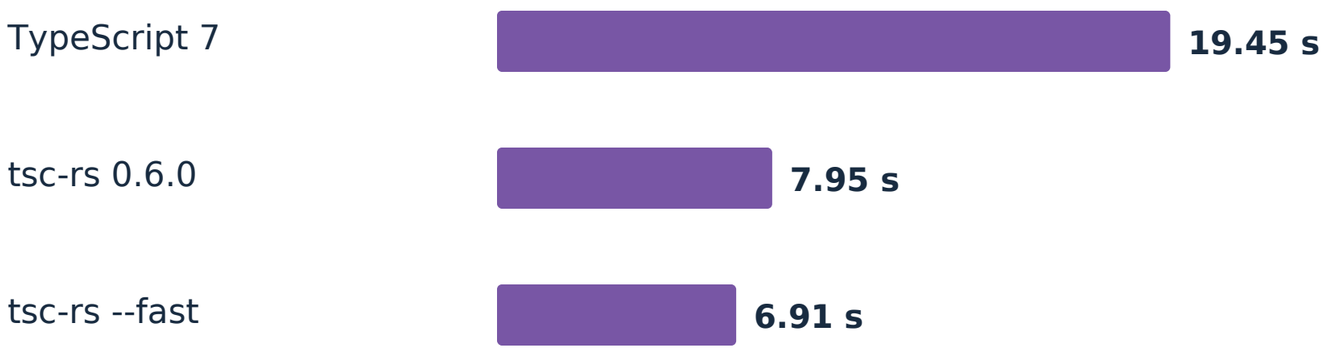
Native median 0.146 ms; guest median 24.04 ms.
Warm state, timing scopes and memory differ from cold.

Warm reuse helps the guest adapter. The known reviewed function is still dramatically cheaper to call natively; fresh VFS had similar median cost.

25 Recorded evidence: native tsc-rs gains

Upstream T3 Code benchmark, 13 October 2026. Native compiler numbers—not in-isolate WASM numbers.

WITHOUT EFFECT / SUM OF FIVE PROJECT MEDIANS



Derived speed ratios vs TS 7: 2.45x default / 2.81x --fast

With Effect diagnostics

TS 7 / Effect: 42.55 s
tsc-rs: 14.30 s; --fast: 9.77 s
Reported: 2.98x / 4.35x faster

Upstream conditions

16 AMD EPYC vCPUs; ~62 GiB RAM.
1 warm-up + 5 runs per project/mode.
Pinned TS 7 parity is reported;
other compiler diagnostics differ.

WASM packaging and constraints

Maintainer: 4.2 MB module (1.8 MB gzip).
Compile module once; new instance per check.
Single-threaded WASM; host stack/ABI probes required.
Native speedup is not inherited automatically.

Wall time != resource reduction

Upstream --fast measurements report CPU time
up to 2.2x and peak memory up to 1.9x.
Measure default and fast under matched budgets;
more checkers are not free parallelism.

Fast native checks support the compiler choice. The decisive missing measurement is this toolchain inside the complete bounded generative loop.

26 Recorded evidence: containment and recovery

Keep successful scoped tests and wider failures in the same evidence picture.

W1/W2 / FRESH-PROCESS GATEWAY CASES

Room A / Room B	937 / 923 ms; 125.2 / 124.0 MiB
Domain packet	984 ms; 131.1 MiB; oracle matched
Infinite loop	2,389 ms; terminal exhausted
Allocation bomb	1,067 ms; guest heap exhausted
Output flood	5,230 ms; external timeout

Flood timed out before proving the transport-cap threshold.

W3/W4 / RETURN AND PROCESS RECOVERY

Source-bound return: 17 checks passed

Two origins; stale/revoked/corrupt handling;
atomic finish + return; no cross-room mutation.

Six controller trials / five Linux methods

SIGKILL before and after permit / payload / spool;
unknown outcomes retained; active cancel verified.
All observed owned groups absent or empty.

Published-checkout test scope

180 tests: 170 executed passes + 10 Linux skips.
Separate native runs covered named methods.
The two-room arithmetic remains synthetic;
visible generative UI was not qualified here.

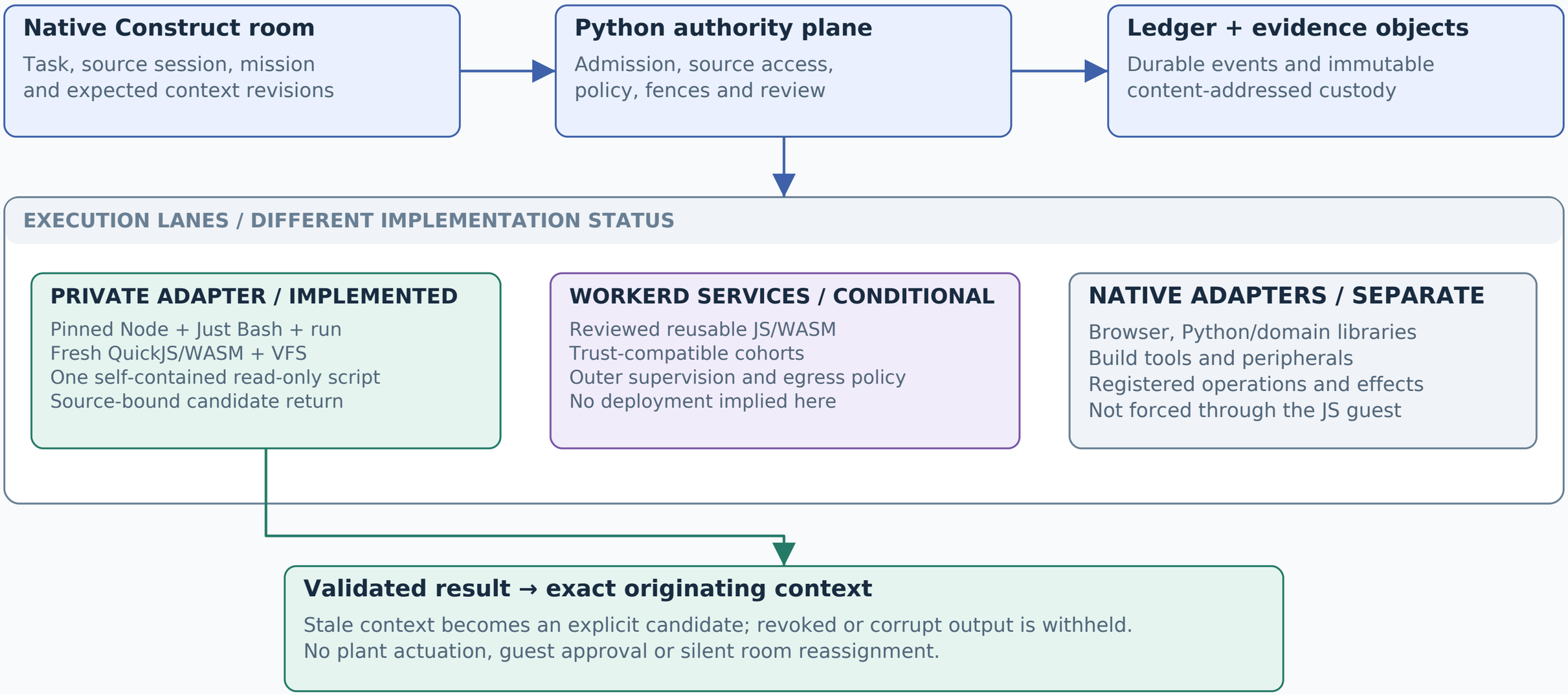
Wider dirty-worktree run remained failed

2,106 tests: 17 failures, 325 errors, 259 skips.
Different and broader artifact; not superseded
by the focused passing suite.
Physical power-loss acceptance remains open.

The private read-only adapter has concrete containment and recovery evidence. It is not a released multi-tenant generative UI / swarm platform.

27 Field OS is the applied case study

The general factory pattern maps onto existing domain owners instead of replacing them.



Field OS demonstrates part of the substrate. The larger concept applies to IDEs, business systems, analysis workbenches and adaptive software.

28 The decisive experiment: the whole local loop

Freeze identical tasks, expected outcomes, toolchains and budgets before comparing architectures.

Matched task set + independent oracle + exact artifact identities

Small generated tool; type-heavy code; data transform; interactive UI; adversarial loop/heap/output cases.

CONTROLLED ARMS / RUN THE SAME USEFUL TASKS

A / Warm native baseline

Vetted function or declarative composition

B / Current guest adapter

Fresh guest and VFS; known existing boundary

C / All-local WASM factory

Checker + guest + loop; matched resource limits

D / Child Worker topology

Loader + candidate V8; account calls and creation

MEASURE

End-to-end and per-stage costs

Cold/warm; p50/p95; accepted count; CPU; memory; model tokens; copies; evaluator and render cost.

HARD ACCEPTANCE GATES

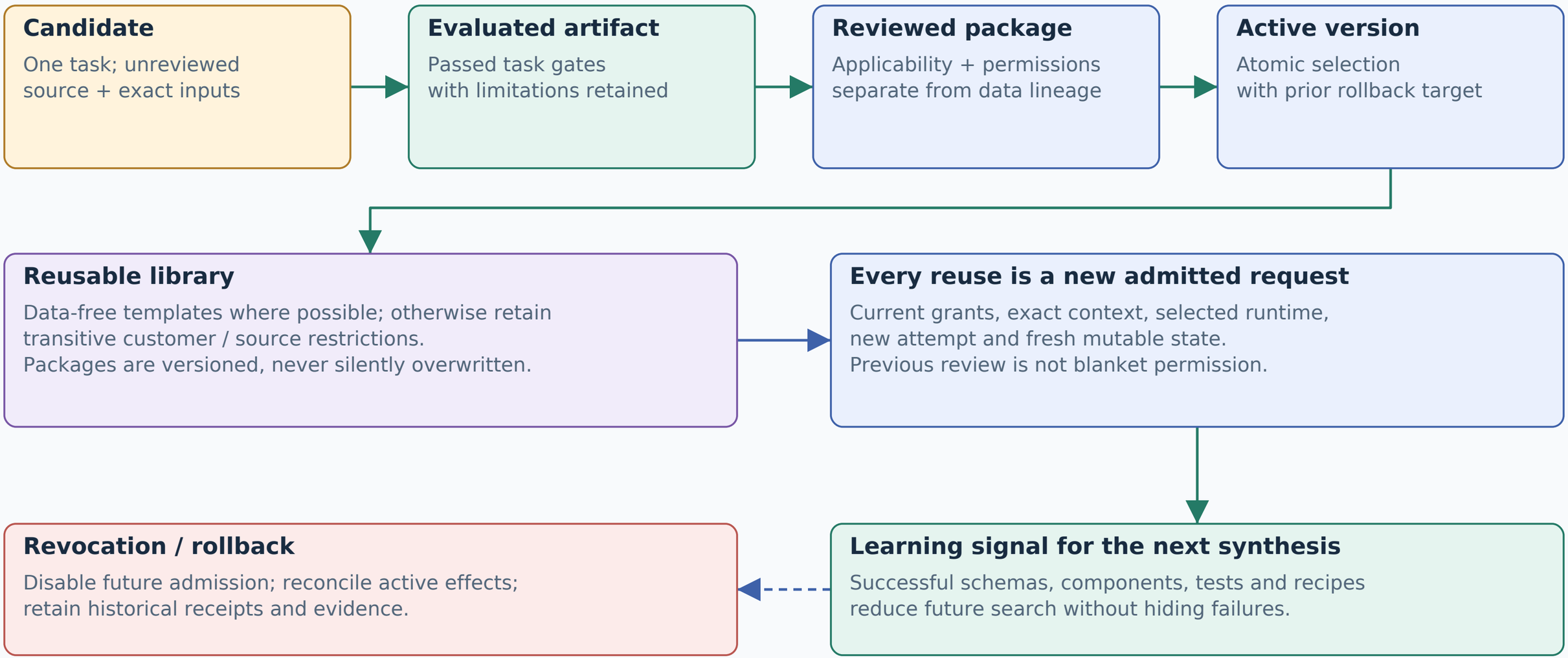
Correctness before speed

No scope leak; bounded closure; diagnostic parity; UI task success; independent held-out pass.

Prove more accepted useful improvements per second and per unit cost—while preserving correctness, isolation and the user experience.

29 From disposable candidate to reusable software

A versioned promotion path turns successful experiments into a growing library of trusted tools.



The compounding asset is an evidence-backed library of useful software—not a collection of unreviewed generated code or cached heaps.

30 The bigger picture—and the evidence frontier

Software synthesis can become a runtime capability, provided the complete feedback loop earns its claims.

ESTABLISHED BUILDING BLOCKS

Recorded local substrate

Warm guest machinery;
fresh VFS and VM;
contained execution and recovery.

Upstream tool capabilities

Fast native Rust checker;
WASM compiler artifact;
runtime-created Workers.

Limits still visible

No local full-loop benchmark;
no proved swarm economy;
no broad appliance acceptance.

NEXT INTEGRATION PROOF

One integrated factory

Pinned WASM checker;
isolated candidate behavior;
real independent measurements.

One working generative UI

Typed behavior + stable state;
useful task completion;
measured improvement loop.

One bounded search campaign

Fixed budget and oracle;
diverse frontier;
accepted outcomes versus baseline.

PLATFORM IMPLICATIONS

Adaptive applications

Temporary tools for a task;
UI generated around intent;
keep useful versions.

Agentic development platforms

Cheaper executable feedback;
more verified search per budget;
less idle environment occupancy.

Reusable software ecosystem

Trusted package library;
portable capability contracts;
local, browser and cloud lanes.

Potentially transformative: task-specific software, generated and improved in place. Demonstrated today: useful components and a bounded applied slice.

Source register / evidence boundaries

Source titles are clickable. Full URLs and exact revisions are also in SOURCES.md and source/sources.json.

R01 Field OS Worker Fabric v1 specification

Defines layered architecture, lifecycle, limits, promotion and acceptance scope.

R03 Worker fabric runtime guide

Actual private adapter, dependencies, limits and declared boundary.

R04 Main-rig execution receipt

Cold/warm tables, oracle scope, retained-state findings and failed model trial.

R05 Private W1/W2 receipt

Seven gateway cases, containment, durability and failed broader suite.

R06 Private W3/W4 receipt

Return/recovery tests, published-checkout scope and retained failures.

R07 Admission and fence implementation

Closed contracts and durable control boundaries.

R08 Supervised execution implementation

Host resource accounting and verified owned execution.

R09 Guest bridge implementation

Fresh guest/VFS behavior and removed host ports.

R11 Operational instrument binding contract

Native UI binding, owner identity, explicit source context and limits.

U01 ts-rust / tsc-rs README and native benchmarks

Pinned 14 October checkout; benchmark measured 13 October 2026. Native 0.6.0, not WASM.

U02 ts-rust WASM interface

Module size, in-memory API, instance reuse/reset, build/publication and stack constraints.

U04 Dynamic Workers: getting started

Runtime loading, supported JS/Python/WASM inputs, and TypeScript precompilation. Checked 14 October 2026.

U05 Dynamic Workers API reference

load/get semantics, content identity and non-guaranteed warm residence. Checked 14 October 2026.

U06 Dynamic Workers platform limits

Four distinct active workers per request; ten per Durable Object I/O context. Checked 14 October 2026.

Recorded ≠ rerun. Upstream native compiler timings ≠ WASM timings. Proposed architecture ≠ deployed capability.

Source register / evidence boundaries

Source titles are clickable. Full URLs and exact revisions are also in SOURCES.md and source/sources.json.

U07 Workers platform limits

Memory accounting includes JS and WASM; per-isolate rather than per-invocation. Checked 14 October 2026.

U08 Dynamic Workers custom resource limits

CPU and subrequest ceilings; lower limits can be supplied per invocation. Checked 14 October 2026.

U09 Workers performance and timers

Production clock restrictions and local measurement difference. Checked 14 October 2026.

U11 Cloudflare Code Mode architecture

Related code-based orchestration, executor/connector/durable-runtime separation. Experimental API; checked 14 October 2026.

U13 workerd README

Runtime design, nanoservices and explicit standalone sandbox warning. Checked 14 October 2026.

U14 Pinned workerd server source

NullIsolateLimitEnforcer and ignored dynamic ResourceLimits at the research pin; do not generalize to uninspected revisions.

Recorded ≠ rerun. Upstream native compiler timings ≠ WASM timings. Proposed architecture ≠ deployed capability.